

Bot de Análise Financeira e Compras

Documentação Técnica

Integrante	RA	Módulo
Eduardo Pereira Silva	202601009	dados/ — entrada e tratamento dos dados
Arthur Lopes Jardim	202600412	motor_analise/ — motor de análise financeira
João Thiago Peixoto Luzine	202601034	fornecedores/ — recomendações de economia
Miguel Cruzeiro Pinheiro	202600428	interface/ — interface e relatórios

Curso: Inteligência Artificial — UFCAT

Disciplina: Introdução à Programação

Trabalho Final — Entrega 3 (Entrega Final + Apresentação na Mostra)

16 de julho de 2026

Repositório: github.com/ArthurLJardim/An-lise-financeira-

2. Introdução

Empresas de pequeno e médio porte recebem, todo período contábil, um **balancete** — o documento que lista o saldo de todas as contas do plano de contas da empresa — mas raramente têm tempo ou ferramenta pra transformar esse documento em uma análise de gastos que ajude a tomar decisão. O balancete fica arquivado, e sinais de gasto fora do padrão (um fornecedor que encareceu, uma categoria que cresceu muito mês a mês) passam despercebidos até virarem um problema maior.

A motivação do projeto é automatizar essa análise: pegar o balancete que a empresa já gera no sistema contábil (sem exigir nenhuma planilha preparada à mão) e devolver, de forma automática, se o período deu lucro ou prejuízo, onde a empresa mais gastou, o que fugiu do esperado e o que pode ser feito a respeito.

Os objetivos do sistema são:

- Ler o balancete contábil diretamente em PDF, sem conversão manual;
- Reconstruir a hierarquia de contas (sintéticas x analíticas) e classificar cada conta em receita ou despesa, por categoria;
- Calcular o resultado do período (lucro/prejuízo/margem) e comparar com um orçamento e/ou histórico;
- Gerar alertas quando um gasto foge do esperado, por severidade;
- Sugerir ações de economia e fornecedores alternativos a partir dos alertas;
- Apresentar tudo em um relatório de texto e/ou em uma interface visual.

3. Descrição do Sistema

O sistema é organizado em quatro módulos independentes, um por integrante do grupo, que se comunicam por um **contrato de dados** comum — um `pandas.DataFrame` de lançamentos financeiros (colunas `data`, `categoria`, `fornecedor`, `valor`, `tipo`), documentado em `docs/CONTRATO_DADOS.md`. Essa separação permite que cada módulo evolua sem quebrar os outros, desde que respeite o formato combinado.



Figura 1 — Fluxograma de funcionamento do sistema.

dados/ (Eduardo)

Lê o balancete em PDF (`ler_balancete_pdf`). Reconstrói a hierarquia entre contas sintéticas (totalizadoras) e analíticas por reconciliação de valores ($\text{saldo anterior} + \text{débito} - \text{crédito} = \text{saldo atual}$), com uma checagem de identidade contábil ($\text{débito total} = \text{crédito total}$) para detectar reconstruções incorretas. Classifica cada conta analítica em receita/despesa e categoria por palavra-chave. Inclui OCR de fallback (Tesseract) para balancetes escaneados.

motor_analise/ (Arthur)

Recebe os lançamentos tratados e calcula o resultado do período (receita, despesa, lucro/prejuízo, margem), rankings de gasto por categoria/fornecedor/produto, variações contra orçamento e/ou histórico, e gera alertas por severidade (baixa/média/alta/crítica). Também monta o relatório em texto, salvo em `relatorios/` e aberto automaticamente no Bloco de Notas.

fornecedores/ (João Thiago)

A partir dos alertas do motor, gera recomendações de ação por categoria (renegociar contrato, cotar fornecedor alternativo, auditar consumo), priorizadas por severidade, com uma estimativa de quanto a

categoria gastou acima da referência.

interface/ (Miguel)

Aplicação em Streamlit que orchestra os módulos acima: upload do balancete, exibição dos metadados extraídos (empresa/CNPJ/período), uma tabela editável para revisar a classificação automática antes de analisar, gráficos de maiores despesas, tabela de alertas e as recomendações de fornecedores.

4. Tecnologias e Bibliotecas Utilizadas

Tecnologia	Uso e justificativa
Python 3	Linguagem base — ecossistema maduro para tratamento de dados e prototipagem rápida.
pandas	Estrutura de DataFrame usada como contrato comum de dados entre todos os módulos.
pypdf	Extração do texto do balancete em PDF, base do parser de layouts contábeis.
pytesseract + pypdfium2	OCR de fallback para balancetes escaneados/sem camada de texto extraível.
streamlit	Interface web funcional sem exigir desenvolvimento de front-end, viável dentro do prazo do projeto.
plotly	Gráficos interativos (maiores despesas por categoria) na interface.
pydantic + fastapi	Validação de dados e API opcional no pipeline ETL mais amplo (Excel/CSV/PDF genérico).
pdfplumber / PyPDF2	Suporte adicional de leitura de PDF no pipeline ETL de tratamento de dados.
pytest	Suíte de 89 testes automatizados validando o parser de balancetes e o motor de análise.

5. Instruções de Instalação e Execução

Requisito único: Python instalado (marcando a opção "Add Python to PATH" no instalador). O repositório completo está em github.com/ArthurLJardim/An-lise-financeira-.

Opção 1 — sem terminal (duplo-clique):

- 1 Dar duplo-clique em `instalar_dependencias.bat` (uma vez só).
- 2 Colocar o balancete (PDF) dentro da pasta entrada/.
- 3 Dar duplo-clique em `rodar_bot.bat` — o relatório é gerado e aberto sozinho no Bloco de Notas.

Opção 2 — terminal:

```
pip install -r requirements.txt
python rodar_bot.py (processa tudo que estiver em entrada/)
python analisar_documento.py balancete.pdf --historico anterior.pdf (mais opções: orçamento, histórico, período)
streamlit run interface/interface.py (interface visual no navegador, local)
```

6. Divisão de Tarefas

Integrante	Entregas principais
Eduardo Pereira Silva	Leitor de balancete em PDF (múltiplos layouts, OCR de fallback), reconstrução de hierarquia de contas e classificação automática por categoria; pipeline ETL genérico (Excel/CSV/PDF).
Arthur Lopes Jardim	Motor de análise financeira: cálculo de resultado, rankings, variações, alertas por severidade e geração do relatório em texto; integração final dos quatro módulos.
João Thiago Peixoto Luzine	Módulo de recomendações: sugestões de ação e economia por categoria a partir dos alertas do motor.

7. Dificuldades e Lições Aprendidas

Layouts diferentes por sistema contábil

A ordem em que o PDF grava as colunas no texto extraído varia de sistema contábil para sistema contábil, e nem sempre é a ordem visual da tabela. Foi preciso desenhar um registro de "layouts" reconhecidos automaticamente por reconciliação de valores (qual ordem de colunas faz saldo anterior + débito – crédito bater com o saldo atual), em vez de assumir um único formato fixo.

Hierarquia de contas não é explícita no PDF

O balancete não indica de forma confiável quais contas são totalizadoras e quais são lançamentos de fato. A reconstrução por reconciliação de valores assume que o balancete lista as contas em pré-ordem — uma suposição que pode falhar silenciosamente. Isso motivou uma checagem adicional de identidade contábil (débito total = crédito total entre as contas-folha identificadas), que falha alto em vez de arriscar um número errado.

Validar com dados reais revelou um bug grave que os testes sintéticos não pegaram

Ao validar o parser contra dois balancetes reais de uma empresa industrial, uma conta de rateio de estoque ('estoque inicial') estava sendo somada como se fosse uma despesa isolada — um valor que é, na verdade, só metade de uma fórmula de custo (estoque inicial + compras – estoque final). O bug transformava um lucro real de cerca de R\$ 790 mil em um prejuízo fictício de mais de R\$ 90 milhões, e só apareceu ao testar contra dados reais, não sintéticos.

Integração entre módulos independentes

Com quatro pessoas desenvolvendo módulos em paralelo, alinhar um contrato de dados comum (nomes de coluna, tipos, categorias padronizadas) documentado à parte foi essencial para juntar tudo sem retrabalho no fim do projeto.

8. Conclusão e Trabalhos Futuros

O sistema cumpre o objetivo proposto: ler um balancete contábil em PDF e produzir, de forma automática, uma análise financeira com resultado do período, rankings de gasto, alertas de desvio e recomendações de economia. A solução foi validada tanto por 89 testes automatizados com dados sintéticos quanto por balancetes reais de duas empresas com plano de contas diferente do exemplo original, o que já revelou e corrigiu limitações que só apareceriam em produção.

Como trabalhos futuros, ficam identificados:

- Cadastrar mais layouts de balancete à medida que aparecerem exemplos reais de outros escritórios/sistemas contábeis;
- Tratar com mais precisão contas de rateio específicas (ex.: créditos de IPI/ICMS sobre compras), que hoje geram uma pequena distorção residual no custo calculado;
- Ampliar o vocabulário de categorias para planos de conta menos usuais;
- Publicar a interface em um serviço de hospedagem gratuito, com acesso via navegador sem instalação local.

9. Referências

- Documentação oficial do pandas — <https://pandas.pydata.org/docs/>
- Documentação oficial do Streamlit — <https://docs.streamlit.io/>
- Documentação do pypdf — <https://pypdf.readthedocs.io/>
- Tesseract OCR — <https://github.com/tesseract-ocr/tesseract>
- Conselho Federal de Contabilidade — normas gerais de contabilidade (referência conceitual para a estrutura de balancete e demonstração de resultado).